

## **Metoda "dziel i zwyciężaj" konstrukcji algorytmów (divide&conquer),**

jako zaawansowany sposób wykorzystania techniki rekursji.

Dany jest problem algorytmiczny Q.

Algorytm rozwiązujący problem Q na danych rozmiaru n (zadanie rozmiaru n) metodą "dziel i zwyciężaj" jest skonstruowany następująco:

- wylicza dane dla pewnej liczby takich samych zadań rozmiaru mniejszego od n ("podzadań"),
- rekurencyjnie wykonuje podzadania,
- wyniki podzadań w odpowiedni sposób przekształca ("scala") tak, aby otrzymać rozwiązanie początkowego zadania rozmiaru n.

Poszczególne logiczne części takiego algorytmu mogą się przeplatać ze sobą.

Np. generowanie kolejnych podzadań może być uzależnione od wyników poprzednich podzadań.

Z ogólnych zasad stosowania rekurencji:

definiowany jest minimalny rozmiar zadania przy którym nie definiuje się podzadań i rozwiązuje go wprost - czyli bez rekurencji.

Analiza algorytmu typu "dziel i zwyciężaj":

na ogół za pomocą równania rekurencyjnego.

Podstawowe narzędzie do rozwiązywania równań rekurencyjnych będzie podane pod koniec tego wykładu.

## Przykład 1: mnożenie liczb binarnych

Dane liczby  $x$ ,  $y$ ,  $n$ -bitowe, obliczyć iloczyn  $x \cdot y$ .

Miara złożoności: liczba dodawań i mnożeń pojedynczych bitów.

Algorytm 1, 'szkolny':

```
      10101
      11001
      ----
      10101
    10101
  10101
-----
 1000001101
```

Złożoność:  $\Theta(n^2)$

## Dziel i zwyciężaj - algorytm Karacuby

*Карачуба А., Офман Ю. Умножение многозначных чисел на автоматах // Доклады Академии Наук СССР, 1962, 145, 2.*

**Karatsuba( $x$ ,  $y$ ,  $n$ )**

// mnożenie liczb  $n$ -bitowych

if  $n=1$  then return  $xy$ ;

$x_L := x / 2^{n/2}$ ; //  $n/2$  najbardziej znaczących bitów  $x$

$x_R := x \% 2^{n/2}$ ; //  $n/2$  najmniej znaczących bitów  $x$

$y_L := y / 2^{n/2}$ ; //  $n/2$  najbardziej znaczących bitów  $y$

$y_R := y \% 2^{n/2}$ ; //  $n/2$  najmniej znaczących bitów  $y$

//  $x = 2^{n/2}x_L + x_R$ ;  $y = 2^{n/2}y_L + y_R$ ;

$u := \text{Karatsuba}(x_L + x_R, y_L + y_R, n/2)$ ;

$v := \text{Karatsuba}(x_L, y_L, n/2)$ ;

$w := \text{Karatsuba}(x_R, y_R, n/2)$ ;

return  $v \cdot 2^n + (u - v - w) \cdot 2^{n/2} + w$ ;

end.

Analiza:

- 3 mnożenia liczb  $n/2$ -bitowych (obliczanie  $u, v, w$ ),
- pewna liczba sumowań i przesunięć (shift),
- równanie rekurencyjne:

$$T(n) = 3 T(n/2) + cn, \quad n > 1,$$

$$T(1) \leq c \quad // \text{ stała } c \text{ pasuje w obu wierszach}$$

$T(n) = ?$

Zał.  $n=2^k$ . Metoda drzewa wywołań rekurencyjnych:

- rozrysuj drzewo wywołań rekurencyjnych,
- zsumuj koszt na każdym poziomie drzewa,
  - na  $i$ -tej głębokości jest  $3^i$  podzadań o wielkości  $n/2^i$ ,  $i=0, \dots, k=\lg n$ ,
  - czas rozwiązania każdego to  $cn/2^i$ ,
  - sumaryczny czas:

$$T(n) = cn((3/2)^0 + (3/2)^1 + (3/2)^2 + \dots + (3/2)^{\lg n});$$

- ze wzoru na sumę ciągu geometrycznego:

$$T(n) = 2c \cdot n^{\lg 3} - 2cn$$

$$= \Theta(n^{\lg 3})$$

$$\approx \Theta(n^{1.58})$$

Złożoność rzędu istotnie mniejszego niż  $n^2$ .

**Uwaga:** pominęliśmy fakt, że w mnożeniu

$$u := (e + f) * (g + h)$$

czynniki są  $n/2+1$  – bitowe (przeniesienie).

- rozbijamy każdy czynnik na część  $n/2$  – bitową i drugą 1-bitową,
- sprowadzamy do mnożenia dwóch liczb  $n/2$ -bitowych plus dodawania i przesunięcia.

**Uwaga:**

najszybszy znany algorytm mnożenia liczb:  
Schönhage-Strassen,  $O(n \log n \log \log n)$ ,  
wykorzystuje szybka transformatę Fouriera.

**Przykład 2: mnożenie macierzy metodą Strassena.**

Dane: macierze kwadratowe  $A, B \in \mathbb{R}^{n \times n}$ ,  $n=2^k$

Wynik:  $C=AB$ ,  $c_{i,j} = \sum_{k=1..n} a_{i,k} * b_{k,j}$ .

Metoda klasyczna: wprost ze wzoru.

Złożoność (mnożenia dominują):  $\Theta(n^3)$

**Metodą dziel i zwyciężaj:**

Każdą macierz traktujemy jako macierz  $2 \times 2$ , każdy z 4 elementów jest podmacierzą kwadratową  $(n/2) \times (n/2)$ .

Oznaczmy podmacierze A przez  $A_{1,1}$ ,  $A_{1,2}$ ,  $A_{2,1}$ ,  $A_{2,2}$ , analogicznie B i C, wykonując działania na podmacierzach otrzymujemy:

$$C_{1,1} = A_{1,1} * B_{1,1} + A_{1,2} * B_{2,1}$$

$$C_{1,2} = A_{1,1} * B_{1,2} + A_{1,2} * B_{2,2}$$

$$C_{2,1} = A_{2,1} * B_{1,1} + A_{2,2} * B_{2,1}$$

$$C_{2,2} = A_{2,1} * B_{1,2} + A_{2,2} * B_{2,2}$$

Algorytm zbudowany na tych wzorach ma złożoność opisaną równaniem:

$$T(n) = 8 T(n/2) + \Theta(n^2),$$

którego rozwiązaniem jest  $T(n)=\Theta(n^3)$ .

Pomysł Strassena:

mnożenie macierzy liczbowych  $2 \times 2$  za pomocą 7  
(a nie 8) mnożeń:

$$p_1 = a_{1,1} * (b_{1,2} - b_{2,2})$$

$$p_2 = (a_{1,1} + a_{1,2}) * b_{2,2}$$

$$p_3 = (a_{2,1} + a_{2,2}) * b_{1,1}$$

$$p_4 = a_{2,2} * (b_{2,1} - b_{1,1})$$

$$p_5 = (a_{1,1} + a_{2,2}) * (b_{1,1} + b_{2,2})$$

$$p_6 = (a_{1,2} - a_{2,2}) * (b_{2,1} + b_{2,2})$$

$$p_7 = (a_{1,1} - a_{2,1}) * (b_{1,1} + b_{1,2}).$$

$$c_{1,1} = p_5 + p_4 - p_2 + p_6$$

$$c_{1,2} = p_1 + p_2$$

$$c_{2,1} = p_3 + p_4$$

$$c_{2,2} = p_5 + p_1 - p_3 - p_7$$

(w sumie 18 dodawań/odejmowań)

*Wzorów tych nie trzeba uczyć się na pamięć, ważna jest idea pomysłu Strassena.*

### **Podstawowy fakt:**

metodę 7 mnożeń, ponieważ nie korzysta ona z przemienności mnożenia liczb, ani z dzielenia liczb, można zastosować do mnożenia (w pierścieniu) macierzy.

Czyli, w powyższych wzorach zastąpić liczby przez (pod)macierze.

**Algorytm:**

jeśli  $n=2$  to wykonaj obliczenia z definicji,  
wpp.

- podziel każdą z macierzy  $A, B$ , na 4 podmacierze rozmiaru  $n/2 \times n/2$ ,
- zastosuj wzory na 7 mnożeń do mnożenia i dodawania-odejmowania podmacierzy,
  - wywołując rekurencyjnie ten algorytm dla operacji mnożenia oraz
  - wykonując iteracyjnie wszystkie operacje dodawania-odejmowania.

Złożoność = liczba operacji arytmetycznych:

$$T(n) = c, \text{ dla } n=2$$

8 mnożeń, 4 dodawania

$$T(n) = 7 \cdot T(n/2) + \Theta(n^2), \text{ dla } n > 2$$

operacje dla macierzy  $n/2 \times n/2$ :

7 mnożeń rekurencyjnie + stała liczba dodawań

Rozwiązanie:

$$\begin{aligned} T(n) &= \Theta(n^{\lg 7}) \\ &\approx \Theta(n^{2.808}) \end{aligned}$$

wynika z twierdzenia MT poniżej.

## **Uwagi praktyczne**

do algorytmów Strassena i Karatsuby:

- efektywniejsze od klasycznych (naiwnych) dopiero dla dużych wartości  $n$  (rzędu kilkaset);
- można zaimplementować w niewielkiej ilości pamięci roboczej (mimo intensywnych wywołań rekurencyjnych).

## **do metody dziel i zwyciężaj w ogólności:**

- niewielkie podzadania (np.  $n < 20$ ) należy rozwiązywać prostymi metodami iteracyjnymi (zatrzymujemy wywołania rekurencyjne);
- różne techniki dla przypadku gdy rozmiar nie jest odpowiednią potęgą (np. dwójki), a algorytm wymaga podzadań jednakowego rozmiaru:
  - na początku powiększyć zadanie do rozmiaru najbliższej odpowiedniej potęgi, uzupełniając elementami neutralnymi (np. zerami dla alg. Strassena) - rząd złożoności nie zwiększa się;
  - albo na każdym poziomie rekurencji, jeśli  $n$  niepodzielne przez parametr liczby podzadań, to zwiększać rozmiar o odpowiednią stałą, wypełniając zerami.

Złożoność problemu mnożenia macierzy:

- najlepszy znany algorytm ma złożoność  $O(n^\omega)$ ,  $\omega \approx 2,3728639$  - Vassilevska-Williams (2011), Le Gall (2014);
- wcześniej m.in. Coppersmith i Winograd (1990), Stothers (2010):  $\omega \approx 2,376$ .

Najlepsze oszacowanie od dołu: oczywiste  $\Omega(n^2)$ .

Istnieje hipoteza że  $\omega=2$ .

## Złożoność algorytmów "dziel i zwyciężaj"

Założenia:

Dla każdego zadania rozmiaru  $n$ , w którym następuje generowanie podzadań:

- $a \geq 1$  – liczba wygenerowanych z tego zadania podzadań,
- $b > 1$  – parametr taki, że rozmiar każdego podzadania wynosi  $\lfloor n/b \rfloor$  lub  $\lceil n/b \rceil$ ,
- $\Theta(n^p)$ ,  $p \geq 0$  – złożoność algorytmu poza wywołaniami rekurencyjnymi (czyli głównego zadania rozmiaru  $n$ , bez wszystkich podzadań).

Równanie rekurencyjne na złożoność algorytmu:

$$T(n) = aT(n/b) + \Theta(n^p), \quad n > c,$$

$$T(n) = \Theta(1), \quad n \leq c$$

**Twierdzenie o rekurencji uniwersalnej (MT: Master Theorem)**, uproszczona wersja.

*Uproszczona jest postać równania:*

$$T(n) = aT(n/b) + \Theta(n^p)$$

Oznaczmy  $\log_b a = d$ . Przy założeniach jak powyżej:

Przypadek (1): jeśli  $d > p$ , to  $T(n) = \Theta(n^d)$ ,

Przypadek (2): jeśli  $d = p$ , to  $T(n) = \Theta(n^d \log n)$ ,

Przypadek (3): jeśli  $d < p$ , to  $T(n) = \Theta(n^p)$ .

Przykłady zastosowania:

Algorytm Karatsuby:

$$T(n) = 3T(n/2) + \Theta(n),$$

$$a=3, \quad b=2, \quad d=\lg 3, \quad p=1, \quad d > p$$

$$\text{przypadek 1: } T(n) = \Theta(n^{\lg 3})$$



BinSearch:

$$T(n) = T(n/2) + c,$$

$$a=1, b=2, d=\lg 1=0, p=0, d=p$$

$$\text{przypadek 2: } T(n) = \Theta(n^0) \log n = \Theta(\log n)$$

Kaskadowe sumowanie tablicy (\*):

$$T(n) = T(n/2) + n,$$

$$a=1, b=2, d=\lg 1=0, p=1, d < p$$

$$\text{przypadek 3: } T(n) = \Theta(n)$$

(\*) najczęściej stosowany jako algorytm równoległy:

- zsumuj  $n/2$  par elementów sąsiednich:  
 $a_1+a_2, a_3+a_4, \dots$ ;
- rekurencyjnie oblicz sumę otrzymanych  $n/2$  wyników.

### **Dowód tw. MT.**

Uprośćmy:  $T(n)=aT(n/b)+n^p, T(1)=1, n=b^k,$

Dla innych wartości  $n$  dowód podobny, bardziej skomplikowany ze względu na użycie podłóg/sufitów.

$$T(b^k) = aT(b^{k-1}) + (b^k)^p,$$

dzielimy przez  $a^k$  :

$$T(b^k)/a^k = T(b^{k-1})/a^{k-1} + (b^p/a)^k,$$

dla  $k:=k-1$ :

$$T(b^{k-1})/a^{k-1} = T(b^{k-2})/a^{k-2} + (b^p/a)^{k-1}, \text{ itd.}$$

...

$$T(b^1)/a^1 = T(b^0)/a^0 + (b^p/a)^1,$$

Sumujemy stronami i redukujemy teleskopowo:

$$T(b^k)/a^k = \sum_{i=0, \dots, k} (b^p/a)^i,$$

Zatem  $T(n) = a^k \sum_{i=0, \dots, k} (b^p/a)^i.$

Zmieniamy podstawę logarytmu:

$$a^k = a^{\log_b n} = a^{\log_b a \log_a n} = n^{\log_b a} = n^d,$$

i otrzymujemy:

$$T(n) = n^d \sum_{i=0, \dots, k} (b^p/a)^i.$$

Identyczny wynik dostajemy analizując a-arne drzewo wywołań rekurencyjnych i sumując koszty na wszystkich poziomach:

- głębokość 0: korzeń, koszt  $(b^k)^p$ ,
- głębokość  $i < k$ :  $a^i$  podzadań rozmiaru  $b^{k-i}$ , koszt  $a^i(b^{k-i})^p$ ;
- głębokość  $k$ :  $a^k = n^d$  liści rozmiaru 1, koszt  $n^d$ .

Przypadki:

(1)  $d > p$ :

- $b^d = a \Rightarrow b^p/a < 1$ ,  
szereg geometryczny zbieżny, suma jest stałą
- $T(n) = \Theta(n^d)$ ;

w przypadku (1) o wielkości  $T(n)$  decyduje liczba liści drzewa wywołań;

(2)  $d = p$ :

- $b^p/a = 1$ ,
- $T(n) = n^d(k+1) = \Theta(n^d \log n)$ ;

tutaj koszty na każdym poziomie drzewa są identyczne;

(3)  $d < p$ :

- obliczamy sumę częściową ciągu geometrycznego, (ćwiczenie) otrzymując
  - $T(n) = \Theta(n^p)$ ;

w przypadku o wielkości  $T(n)$  decyduje koszt korzenia.

## Twierdzenie MT - wersja z logarytmem

Niech  $a \geq 1$ ,  $b > 1$ ,  $p, q \geq 0$ ,

$$T(n) = aT(n/b) + \Theta(n^p \log^q n), n > c,$$

$$T(n) = \Theta(1), n \leq c$$

Oznaczmy  $\log_b a = d$ . Wówczas:

(1) jeśli  $d > p$ , to  $T(n) = \Theta(n^d)$ ,

(2) jeśli  $d = p$ , to  $T(n) = \Theta(n^d \log^{q+1} n)$ ,

(3) jeśli  $d < p$ , to  $T(n) = \Theta(n^p \log^q n)$

*bez dowodu*

Przykład: sortowanie bitoniczne

$$T(n) = 2T(n/2) + \Theta(n \log n);$$

$$a=b=2, d=\lg 2=1, p=1, q=1 - \text{przypadek 2}$$

$$T(n) = \Theta(n \log^2 n).$$

## Twierdzenie MT - wersja liniowa

Jeśli  $c_i > 0$ ,  $i=1, \dots, m$ ,  $\sum_{i=1, \dots, m} c_i < 1$ , to rozwiązanie równania

$$T(n) = \sum_{i=1, \dots, m} T(c_i n) + \Theta(n)$$

wynosi  $T(n) = \Theta(n)$ .

*bez dowodu*

Przykład: algorytm "mediana median" ("magiczne piątki") znajdowania k-tego elementu:

$$T(n) = T(n/5) + T(3n/4) + \Theta(n);$$

$$1/5 + 3/4 = 19/20 < 1$$

$$T(n) = \Theta(n).$$